

Reviewer Pack — Minimal Order of Local Units in Adjoint Group and Communicat...

Entry e005c8be80a3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 15:18:48 UTC

Minimal Order of Local Units in Adjoint Group and Communication Complexity Rank Variance Bound

Entry ID: e005c8be80a3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 15:18:48 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Adjoint Representation Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For any given n -ary communication protocol, the minimal order of local units in the adjoint group corresponding to the protocol's information matrix is linearly correlated with its rank variance, such that $O(n^c) \leq \text{rank_variance} \leq \Theta(n^{\{1+c\}})$ for some constant c .

Rationale (proposer's reasoning):

The adjoint representation theory provides a structured way to encode communication protocols as groups. This conjecture bridges group-theoretic invariants to communication complexity by proposing a quantitative relationship between the minimal order of local units and the rank variance, which could expose deeper connections between these fields.

Taxonomy category: Adjoint_Representation_Theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a3bd286449520bc5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal order of local units and rank variance for all protocols is greater than or equal to 0.8, with a mean rank variance within the range $[O(n^c), \Theta(n^{\{1+c\}})]$ for some constant c .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - adjoint representation theory AND communication complexity rank variance - minimal order local units adjoint group AND information matrix rank variance - group theory adjoint group communication complexity AND rank variance bound

Top relevant hits considered: - [<http://arxiv.org/abs/1403.8106v1>] Recent advances on the log-rank conjecture in communication complexity - [<http://arxiv.org/abs/1102.2932v2>] Applications of Monotone Rank to Complexity Theory - [<http://arxiv.org/abs/1611.00827v2>] Geometric complexity theory and matrix powering - [<http://arxiv.org/abs/1908.06409v1>] Schur multipliers of special p -groups of rank 2 - [<http://arxiv.org/abs/2012.00950v4>] The Geometry and Kinematics of the Matrix Lie Group $SE_K(3)$ - [<http://arxiv.org/abs/1304.2668v2>] Andrews-Curtis and Nielsen equivalence relations on some infinite groups - [<http://arxiv.org/abs/hep-ph/0610012v1>] Tevatron-for-LHC Report of the QCD Working Group - [<http://arxiv.org/abs/math/0608534v2>] On Automorphisms of Finite p -groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_communication_protocol(n):
        # Generate a random n-ary communication protocol
        return [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    def compute_information_matrix(protocol):
        # Compute the information matrix from the protocol
        n = len(protocol)
        info_matrix = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i, n):
                count = sum(1 for k in range(n) if protocol[i][k] != protocol[j][k])
                info_matrix[i][j] = info_matrix[j][i] = Fraction(count, n)
        return info_matrix

    def rank_variance(matrix):
        # Compute the rank variance of the matrix
        n = len(matrix)
```

```

det = determinant(matrix)
if det == 0:
    return float('inf')
rank = gaussian_elimination(matrix)
return (n - rank) / n

def gaussian_elimination(matrix):
    # Perform Gaussian elimination to find the rank of the matrix
    n = len(matrix)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        if matrix[i][i] == 0:
            return float('inf')
        for j in range(i+1, n):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
    rank = sum(1 for row in matrix if any(x != 0 for x in row))
    return rank

def determinant(matrix):
    # Compute the determinant of a square matrix
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    det = 0
    sign = 1
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in matrix[1:]]
        det += sign * matrix[0][i] * determinant(submatrix)
        sign *= -1
    return det

def minimal_order_of_local_units(matrix):
    # Compute the minimal order of local units in the adjoint group
    n = len(matrix)
    adjoint_group = []
    for i in range(n):
        for j in range(i, n):
            if matrix[i][j] != 0:
                adjoint_group.append((i, j))
    return min(abs(x - y) for x, y in adjoint_group)

def pearson_correlation_coefficient(x, y):
    # Compute the Pearson correlation coefficient
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov_xy = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    std_x = math.sqrt(sum((x[i] - mean_x)**2 for i in range(n)) / n)
    std_y = math.sqrt(sum((y[i] - mean_y)**2 for i in range(n)) / n)
    return cov_xy / (std_x * std_y)

```

```

results = []
for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    protocol = generate_communication_protocol(n)
    info_matrix = compute_information_matrix(protocol)
    rank_var = rank_variance(info_matrix)
    min_order_units = minimal_order_of_local_units(info_matrix)
    results.append((min_order_units, rank_var))

x = [res[0] for res in results]
y = [res[1] for res in results]
correlation_coefficient = pearson_correlation_coefficient(x, y)

return {
    "metric_name": "Pearson Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n for _, _ in results),
    "conjecture_holds": correlation_coefficient >= 0.8,
    "counterexample": "" if correlation_coefficient >= 0.8 else "Pearson correlation coefficient < 0.8"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(res["metric_value"] for res in results) / len(results)
    std_metric_value = math.sqrt(sum((res["metric_value"] - mean_metric_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Pearson correlation coefficient < 0.8\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to compute the Pearson correlation coefficient and mean rank variance required to support or falsify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14576
2	propose	ollama_remote	glm4:latest	0	0	13094
3	propose	ollama_remote	glm4:latest	0	0	14426
4	propose	ollama_remote	glm4:latest	0	0	21635
5	preregistration	ollama_remote	glm4:latest	0	0	10007
6	novelty	ollama_remote	glm4:latest	0	0	8595
7	novelty	ollama_remote	glm4:latest	0	0	10484
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	55312
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13895
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13180
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15066
12	judge	ollama_remote	glm4:latest	0	0	23788

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 214057 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in research/audit/e005c8be80a3.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/e005c8be80a3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/e005c8be80a3.tar.gz (if generated)